

Student

I've heard about decorators before — I know `@staticmethod` and `@property`. But I've also seen things like `@retry(max_attempts=3)` with parentheses after the name. What's going on there?

ChatGPT • AI Assistant

CONCEPT CARD

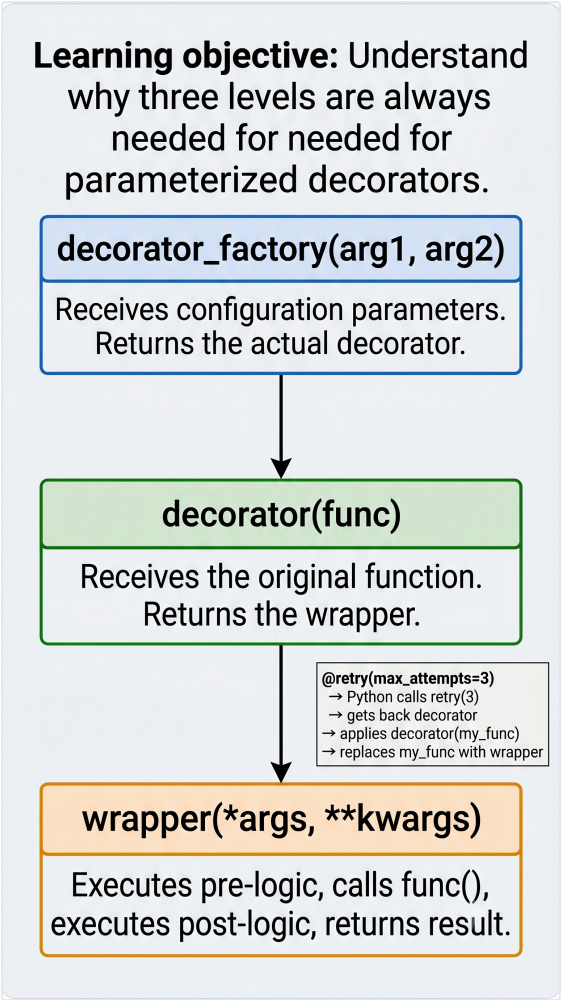
Great question! A plain decorator like `@mydecorator` takes one argument — the function it wraps. But a decorator with arguments, like `@retry(max_attempts=3)`, is actually a **decorator factory** — a function that *returns* a decorator. The parentheses after the name call the factory first, and what comes back is the actual decorator applied to your function. It's three levels of nesting: the factory gets the config, the decorator gets the function, and the wrapper does the actual work.

Student

Three levels of nesting sounds scary. Can you show me why a plain decorator can't just accept arguments directly?

ChatGPT • AI Assistant

Absolutely — let's start by feeling the pain first.



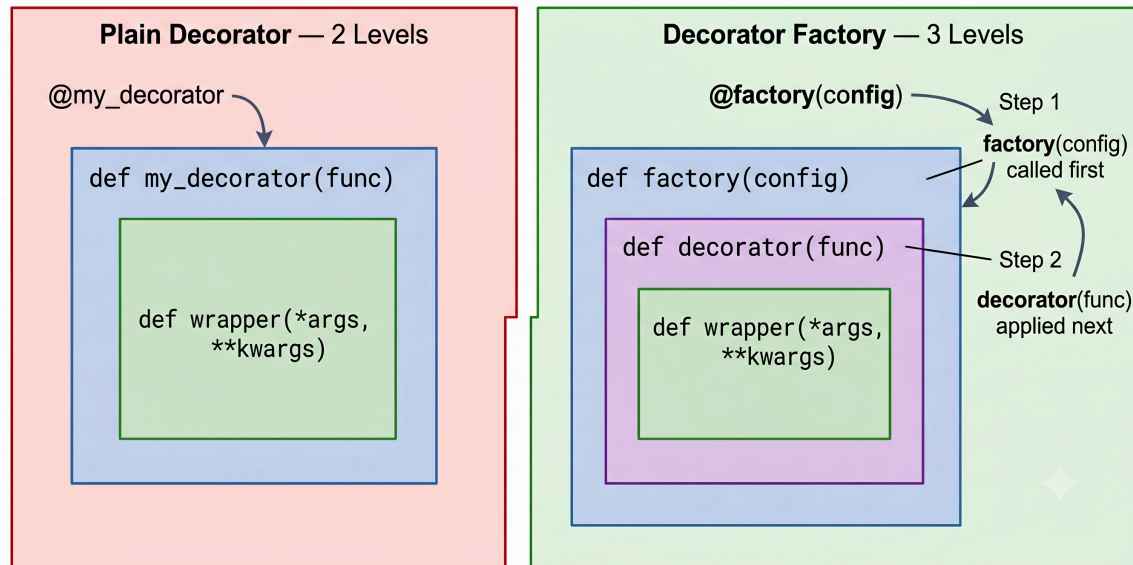
Decorator Factory: Three-Level Nesting Model

Plain Decorator vs Decorator Factory — At a Glance		
Feature	Plain Decorator	Decorator Factory
Syntax	@my_decorator	@my_decorator(arg)
Nesting levels	2 (decorator + wrapper)	3 (factory + decorator + wrapper)
Accepts config?	No	Yes
functools.wraps goes on	wrapper	innermost wrapper
Common use cases	logging, timing	retry, rate-limit, validate
Called at import time?	Yes, immediately	Yes — factory called first, then decorator applied

WHY PLAIN DECORATORS CANNOT ACCEPT ARGUMENTS

ChatGPT • AI Assistant

Learning objective: Visualize why an extra level is needed to accommodate configuration arguments.



Two-Level vs Three-Level Decorator Nesting

Student

Okay, show me the pain — what happens if I try to pass arguments to a plain decorator?

ChatGPT • AI Assistant

EXAMPLE CARD

Example 1 — Basic: The broken attempt

python — Code Example

```
# WRONG — This will fail!
def repeat(func, times): # Two parameters: func AND times
    def wrapper(*args, **kwargs):
        for _ in range(times):
            func(*args, **kwargs)
```

```

    return wrapper

@repeat(times=3) # Python tries: repeat(times=3) → gets a partial
call, not a decorator!
def greet():
    print('Hello!')

# TypeError: repeat() missing 1 required positional argument: 'func'

```

Here's the key insight: when Python sees `@repeat(times=3)`, it evaluates `repeat(times=3)` **first**. But `repeat` expects `func` as its first argument! Python doesn't know you want to pass `func` separately later. The decorator protocol is rigid: `@something` means Python calls `something(your_function)`. If `something` is `repeat(times=3)`, then `repeat(times=3)` must *return* a callable that accepts one argument — the function.

Student

So the only fix is to add another level of nesting?

ChatGPT • AI Assistant

Exactly! Let's build it correctly step by step.

EXAMPLE CARD

Example 2 — Intermediate: The correct structure

python – Code Example

```

import functools

# Level 1: The factory – receives configuration
def repeat(times):          # ← gets the argument
    # Level 2: The decorator – receives the function
    def decorator(func):    # ← gets the original function
        # Level 3: The wrapper – does the actual work
        @functools.wraps(func) # MUST be here, on the wrapper
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator        # factory returns the decorator

```

```
@repeat(times=3)
def greet(name):
    print(f'Hello, {name}!')

greet('Srikanth')
# Output:
# Hello, Srikanth!
# Hello, Srikanth!
# Hello, Srikanth!
```

Student

Why does `@functools.wraps(func)` go on the innermost wrapper and not the middle function?

ChatGPT • AI Assistant

`functools.wraps(func)` copies the original function's metadata — its `__name__`, `__doc__`, `__module__` — onto the wrapper. The wrapper is the function that *replaces* your original function after decoration. The middle `decorator` function is just scaffolding — nobody calls it directly. If you put `wraps` on `decorator`, you'd be copying metadata onto something Python throws away. Always put it on the function that actually does the work and gets returned.

❗ COMMON PITFALL / MISTAKE

✗ Incorrect Way:

python – Code Example

```
def repeat(times):
    @functools.wraps # Wrong level!
    def decorator(func):
        def wrapper(*args, **kwargs):
            ...
        return wrapper
    return decorator
```

✓ Correct Way:

python – Code Example

```
def repeat(times):  
    def decorator(func):  
        @functools.wraps(func) # Correct: on the wrapper  
        def wrapper(*args, **kwargs):  
            ...  
        return wrapper  
    return decorator
```

🔗 INTERVIEW CARD

- **Frequency:** Very common in senior Python interviews
- **Question:** 'Why does a parameterized decorator need three levels of nesting instead of two?'
- **Expected Answer:** Because when Python evaluates `@decorator(arg)`, it first calls `decorator(arg)` which must return a callable. That returned callable is then applied to the function. So `decorator(arg)` = factory call returning the real decorator, which then wraps the function.
- **Follow-up:** 'Where exactly must `functools.wraps` be placed, and why?'

🔗 PRACTICE QUIZ

Concept Questions:

1. What does Python do first when it sees `@my_decorator(config)` above a function definition?
2. How many nested functions are required for a decorator that accepts arguments?
3. What metadata does `functools.wraps` preserve?

Output Prediction Questions:

1. What is the output of this code?

python – Code Example

```
def tag(html_tag):  
    def decorator(func):  
        @functools.wraps(func)  
        def wrapper(*args, **kwargs):  
            return f'<{html_tag}>{func(*args, **kwargs)}'  
        return wrapper  
    return decorator
```

```
@tag('b')
def greet(name):
    return f'Hello {name}'

print(greet('Ada'))
```

1. What is `greet.__name__` after the decoration above?

Debugging Question:

1. This code raises `TypeError` — find the bug:

python – Code Example

```
def log(func, level='INFO'):
    def wrapper(*args, **kwargs):
        print(f'[{level}] Calling {func.__name__}')
        return func(*args, **kwargs)
    return wrapper

@log(level='DEBUG')
def process():
    pass
```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Python evaluates `my_decorator(config)` first, which must return a callable (the actual decorator).
2. Three levels: factory → decorator → wrapper.
3. `__name__` , `__doc__` , `__module__` , `__qualname__` , `__annotations__` , `__dict__` , `__wrapped__` .
4. `<b>Hello Ada</b>`
5. `greet` (because `functools.wraps` preserved the name)
6. Bug: `log` takes `func` as first argument — so `@log(level='DEBUG')` tries to call `log(level='DEBUG')` with no `func` . Fix: `def log(func, level='INFO'):` at the outer level, `def decorator(func):` at the middle level.

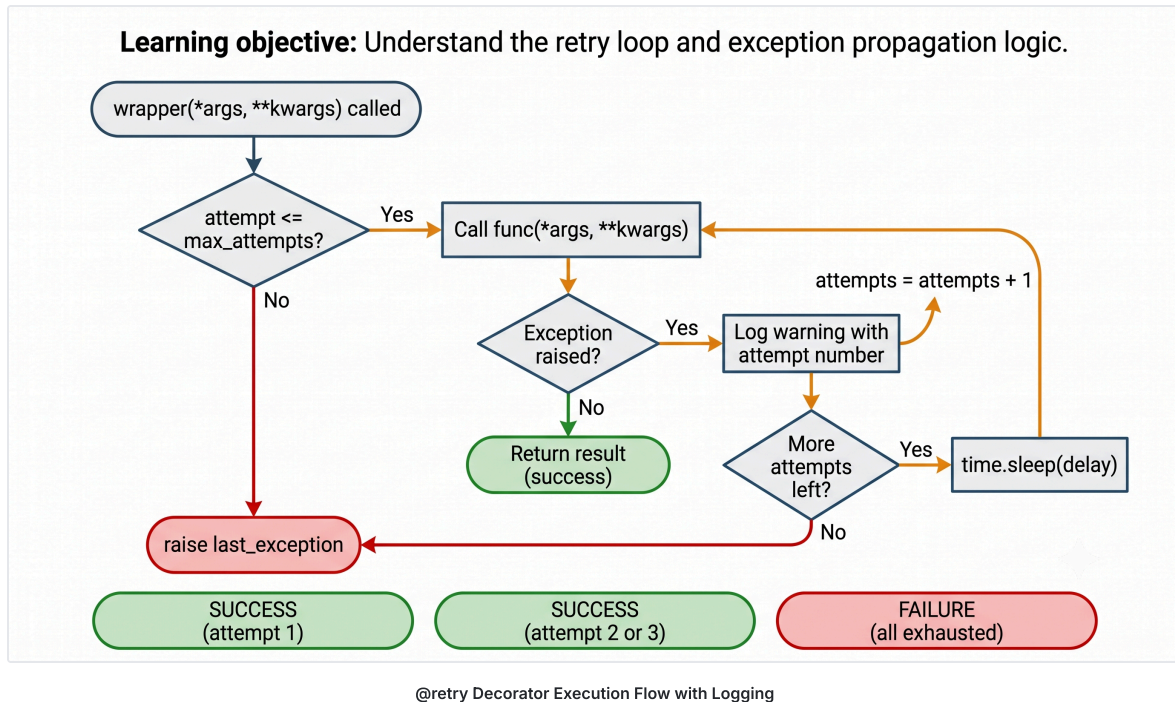
❓ CHECKPOINT QUESTION

Before moving on: Can you explain, in your own words, why

`@retry(max_attempts=3)` requires the outer `retry` function to return a decorator rather than being the decorator itself?

BUILDING THE @RETRY DECORATOR WITH DELAY AND LOGGING

ChatGPT • AI Assistant



Student

This is starting to make sense. Can we build something real? I've heard that a `@retry` decorator is a classic example. Can you show me one that actually retries a failing function?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Yes! Retry logic is one of the most common patterns in production Python. Any time you call an external API, a database, or a network resource, transient failures

happen. A `@retry` decorator handles them transparently without polluting your business logic with loops and error-handling boilerplate.

EXAMPLE CARD

Example 3 — Practical: Full `@retry` decorator

python — Code Example

```
import functools
import time
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def retry(max_attempts=3, delay=1, exceptions=(Exception,)):
    """
    Decorator factory: retries a function on failure.
    :param max_attempts: how many times to try (including first
    attempt)
    :param delay: seconds to wait between retries
    :param exceptions: tuple of exception types to catch
    """
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(1, max_attempts + 1):
                try:
                    result = func(*args, **kwargs)
                    if attempt > 1:
                        logger.info(f'[RETRY SUCCESS] {func.__name__}
succeeded on attempt {attempt}')
                    return result
                except exceptions as e:
                    last_exception = e
                    logger.warning(
                        f'[RETRY] {func.__name__} failed on attempt
{attempt}/{max_attempts}: {e}'
                    )
                    if attempt < max_attempts:
                        time.sleep(delay)
            # All attempts exhausted – re-raise the last exception
            raise last_exception
```

```
return wrapper
return decorator
```

Student

How do I use this?

ChatGPT • AI Assistant

EXAMPLE CARD

Example 4 — Real Business Example: Simulating a flaky API call

python — Code Example

```
import random

@retry(max_attempts=3, delay=0.5, exceptions=(ConnectionError,
TimeoutError))
def fetch_user_data(user_id):
    """Simulates a network call that sometimes fails."""
    if random.random() < 0.6: # 60% chance of failure for demo
        raise ConnectionError(f'Network timeout for user {user_id}')
    return {'id': user_id, 'name': 'Srikanth', 'role': 'engineer'}

# Try it:
try:
    data = fetch_user_data(42)
    print(f'Got data: {data}')
except ConnectionError as e:
    print(f'All retries exhausted: {e}')

# Possible output (varies with random):
# WARNING:__main__: [RETRY] fetch_user_data failed on attempt 1/3:
# Network timeout for user 42
# WARNING:__main__: [RETRY] fetch_user_data failed on attempt 2/3:
# Network timeout for user 42
# INFO:__main__: [RETRY SUCCESS] fetch_user_data succeeded on attempt 3
# Got data: {'id': 42, 'name': 'Srikanth', 'role': 'engineer'}
```

Student

Why do you pass `exceptions=(Exception,)` as a tuple?

ChatGPT • AI Assistant

Because Python's `except` clause with a variable requires either a single exception class or a **tuple** of exception classes. Writing `except exceptions` when `exceptions` is a tuple works perfectly. This design lets callers configure *which* exceptions to retry — you might want to retry `ConnectionError` but not `ValueError`, because a `ValueError` means your input data is wrong and retrying won't fix it.

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — retrying too broadly:

python – Code Example

```
@retry(max_attempts=5) # catches ALL exceptions including ValueError,
                        # TypeError ...
def parse_and_save(data):
    user = User(**data) # If data schema is wrong, this throws
                        # ValueError every time
    db.save(user)       # You'll retry 5 times for a bug that never
                        # heals!
```

✓ Correct — retry only transient errors:

python – Code Example

```
@retry(max_attempts=3, delay=1, exceptions=(ConnectionError,
                                             TimeoutError))
def parse_and_save(data):
    user = User(**data)
    db.save(user)
```

Student

Is this asked in interviews?

ChatGPT • AI Assistant

INTERVIEW CARD

- **Frequency:** High — asked in backend, DevOps-adjacent, and API integration roles
- **Question:** 'Implement a decorator that retries a function N times with exponential backoff.'
- **Expected Answer:** Show the three-level nesting pattern. Mention catching specific exceptions only. Mention `functools.wraps`. Bonus: implement exponential backoff with `time.sleep(delay * (2 ** (attempt - 1)))`.
- **Follow-up:** 'How would you add a jitter (random delay) to avoid thundering herd problems?'

EXERCISE

Modify the `@retry` decorator above to:

1. Use **exponential backoff** — delay doubles after each retry: `delay * 2^(attempt-1)`
2. Add a `max_delay` cap so the delay never exceeds 30 seconds
3. Log the next retry delay in each warning message

Expected behavior:

code — Code Example

```
Retry 1 failed — waiting 1s before retry 2
Retry 2 failed — waiting 2s before retry 3
Retry 3 failed — waiting 4s before retry 4
```

AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT:

"Act as a senior Python backend engineer. Teach me decorator factories through 10 progressively harder exercises. Start from a simple `@logcalls(level)` decorator and work up to a `@circuitbreaker(failurethreshold, recoverytimeout)` pattern used in microservices. After each exercise: (1) ask me to predict the output, (2) show the solution, (3) explain why this pattern exists in production, (4) give me one interview question about it. Do not proceed until I answer."

PRACTICE QUIZ

Concept Questions:

1. Why should the `exceptions` parameter in `@retry` be a tuple rather than a single exception class?
2. What happens if all retry attempts fail? What should the decorator do?
3. Why would you NOT want `@retry` to catch `ValueError` ?

Output Prediction Questions:

1. How many times does `my_func` get called in total when `@retry(max_attempts=3)` is used and it fails every time?
2. After this decoration, what does `fetch_user_data.__name__` return, and why does that matter?

Debugging Question:

1. Find the bug:

python – Code Example

```
def retry(max_attempts=3):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception:
                    pass
            return wrapper
        return decorator
```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Python's `except clause` requires a tuple for multiple exception types. Passing a tuple also allows callers to specify different exception subsets.
2. After all retries fail, the decorator should re-raise the last exception — never silently swallow it.
3. `ValueError` means the input data is invalid — retrying the same bad input will fail every time. Only transient errors (network, timeout) benefit from retrying.
4. Exactly 3 times (the `max_attempts`).
5. `fetch_user_data` — because `functools.wraps` preserves `__name__` . This matters for logging, debugging, and tools that inspect function metadata.

6. Bug: after all attempts fail, the function silently returns `None` instead of raising. `pass` on the last attempt swallows the exception. Fix: track `last_exception` and re-raise after the loop.

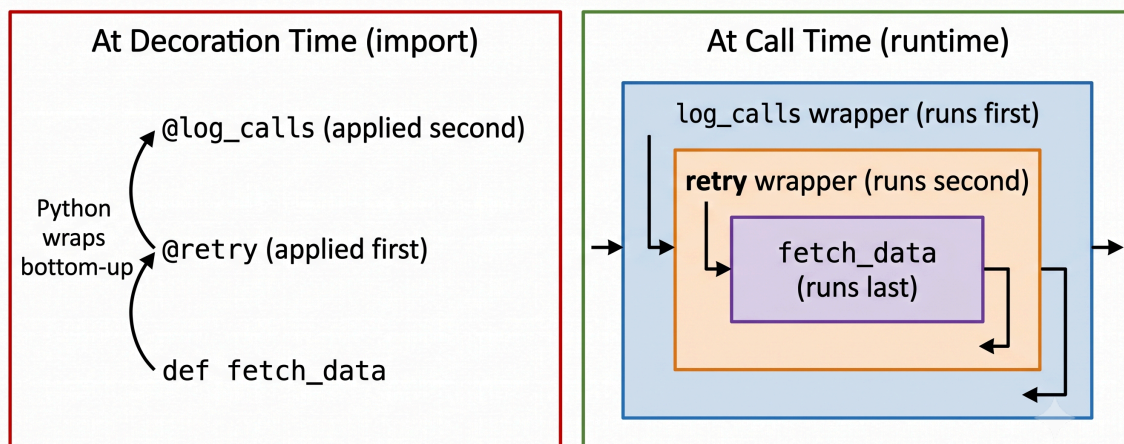
❓ CHECKPOINT QUESTION

Can you explain what exponential backoff is, and why a retry decorator should prefer it over a fixed delay?

STACKING DECORATORS AND DECORATOR ORDER

ChatGPT • AI Assistant

Learning objective: Understand the difference between decoration order and execution order.



Think: Onion model — outermost layer executed first on the way in, last on the way out.

Stacked Decorator Execution Order — Bottom-Up Decoration, Top-Down Execution

Student

Can you stack multiple parameterized decorators on one function? What order do they apply?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — you can stack decorators. Python applies them **bottom-up** during decoration (closest to the function first), but **top-down** during execution (the outermost wrapper runs first when you call the function).

EXAMPLE CARD

Example 5 — Advanced: Stacking @retry and @log_calls

python — Code Example

```
import functools
import time

def log_calls(prefix='CALL'):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            print(f'[{prefix}] Starting {func.__name__}')
            result = func(*args, **kwargs)
            print(f'[{prefix}] Finished {func.__name__}')
            return result
        return wrapper
    return decorator

# Stack order matters!
@log_calls(prefix='API')    # Applied SECOND (outer) – executes FIRST
@retry(max_attempts=2, delay=0) # Applied FIRST (inner) – executes SECOND

def fetch_data(endpoint):
    if endpoint == '/bad':
        raise ConnectionError('Bad endpoint')
    return f'Data from {endpoint}'

fetch_data('/users')

# Output:
# [API] Starting fetch_data           ← log_calls wrapper runs first
# [API] Finished fetch_data          ← log_calls wrapper finishes last
# (retry logic runs transparently inside)
```

Student

So if I swap the decorator order, the behavior changes?

Exactly right. If `@retry` is on top (outer), it retries the *already-logged* function call, meaning you'd see log messages for each retry. If `@log_calls` is on top (outer), it logs once and the retry happens silently inside. Neither is wrong — it depends on what you want to observe.

🔗 ANALOGY

Think of it like wrapping a gift. The decorator closest to the function is the innermost wrapping paper. When someone opens it, they remove the outermost wrapper first. Decoration order (bottom-up) ≠ execution order (top-down).

⚠️ COMMON PITFALL / MISTAKE

✗ Confusing decoration order with execution order:

python – Code Example

```
# Developers often think: 'the top decorator runs first during
decoration'
# Wrong! Bottom decorator is applied first during decoration.
@A # Applied second, executes first (outermost wrapper)
@B # Applied first, executes second (innermost wrapper)
def func(): pass
# This is equivalent to: func = A(B(func))
```

✓ Remember the rule:

'Decoration: bottom-up. Execution: top-down.'

📇 INTERVIEW CARD

- **Frequency:** Medium — asked when candidates claim deep Python knowledge
- **Question:** 'If I have `@A` on top and `@B` below it on a function, which decorator is applied first? Which executes first when I call the function?'
- **Expected Answer:** `@B` is applied first (it wraps the raw function). `@A` is applied second (it wraps B's result). During a call, `@A` 's wrapper runs first, then calls `@B` 's wrapper, which calls the original function.
- **Follow-up:** 'How would you verify this empirically?'

🔥 EXERCISE

Write two decorator factories:

1. `@timer` — prints how long the function took in milliseconds
2. `@validate_positive` — raises `ValueError` if the first argument is negative

Then stack them on a function `compute(n)` that returns `n * n`. Test with `compute(5)` and `compute(-3)`. Try both stacking orders and observe the difference.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT:

"I know how to write decorator factories in Python. Now show me 5 real-world decorator patterns used in production codebases: `@cache` with TTL, `@ratelimit`, `@circuitbreaker`, `@requireauth`, `@validateschema`. For each one: (1) explain the business problem it solves, (2) show a minimal but complete implementation, (3) show a realistic usage example, (4) ask me one interview question about it before showing the answer."

🔍 PRACTICE QUIZ

Concept Questions:

1. If decorators are stacked as `@A` (top) and `@B` (bottom), what is the equivalent single-line expression?
2. Why might the ORDER of `@retry` and `@log_calls` matter?
3. What property of the function is most important for `@timer` to measure accurately?

Output Prediction Questions:

1. What does this print?

python – Code Example

```
def mark(label):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            print(f'>>> {label}')
            return func(*args, **kwargs)
        return wrapper
    return decorator

@mark('OUTER')
@mark('INNER')
def hello():
    print('Hello!')
```

```
hello()
```

1. What is `hello.__name__` after this decoration?

Debugging Question:

1. A developer wants `@timer` to measure only the business logic, not the validation. But after stacking `@timer` on top of `@validate`, the timer includes validation time too. How should they reorder the decorators?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `func = A(B(func))`
2. With `@log_calls` on top: you see one log entry regardless of retries. With `@retry` on top: you see a log entry for *each* attempt, giving more visibility.
3. The time measurement must wrap the actual `func()` call precisely — start timer before `func()`, stop after.
4. Output: `>>> OUTER` then `>>> INNER` then `Hello!`
5. `hello` (`functools.wraps` preserves it at each level)
6. Put `@timer` below `@validate` (i.e., `@validate` on top, `@timer` closer to the function). The timer then wraps only the raw function, not the validation wrapper.

SUMMARY CARD

- A decorator factory requires 3 levels: factory → decorator → wrapper
- `functools.wraps` MUST go on the innermost wrapper
- Decoration order is bottom-up; execution order is top-down
- Catch only specific exception types in `@retry`, never broad `Exception` for logic errors
- Prefer exponential backoff over fixed delay in network retry scenarios
- Stacking decorators changes behavior — always test both orders

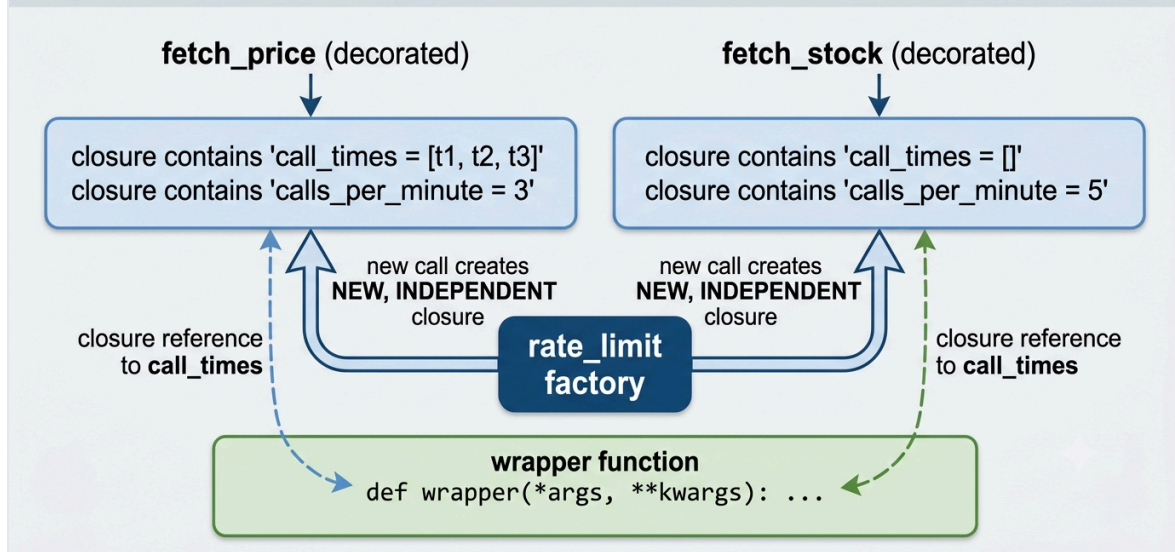
🔍 CHECKPOINT QUESTION

Final challenge: Can you write a `@require_role(role)` decorator factory that checks if a `current_user.role` matches the required role, and raises `PermissionError` if not? Where would you put `functools.wraps`?

PRACTICE PROBLEMS: @RATE_LIMIT AND @VALIDATE_TYPES

ChatGPT • AI Assistant

Learning objective: Understand that each decoration creates an independent closure state — two functions decorated with the same factory do not share counters.



Closure State in Decorator Factories — Shared Counter Model

Student

Okay, I want to try building some on my own. What are the practice problems?

ChatGPT • AI Assistant

🔥 EXERCISE

Practice Problem 1 — Write `@rate_limit(calls_per_minute)`

Requirements:

- Accept `calls_per_minute` as the parameter
- Track how many times the function has been called in the current minute
- If the limit is exceeded, raise a custom `RateLimitError` with a message like `'Rate limit exceeded: max 5 calls per minute for fetch_data'`
- Reset the counter every 60 seconds

Starter template:

python — Code Example

```
import time
import functools

class RateLimitError(Exception):
    pass

def rate_limit(calls_per_minute):
    def decorator(func):
        # Your state needs to live HERE (in the closure)
        call_times = [] # Track timestamps of recent calls

        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            now = time.time()
            # TODO: Remove calls older than 60 seconds from call_times
            # TODO: Check if len(call_times) ≥ calls_per_minute
            # TODO: If yes, raise RateLimitError
            # TODO: Append current time to call_times
            # TODO: Call and return func(*args, **kwargs)
            pass

        return wrapper
    return decorator

# Test it:
@rate_limit(calls_per_minute=3)
def fetch_price(ticker):
    return f'Price of {ticker}: $100'

for i in range(5):
    try:
        print(fetch_price('AAPL'))
    except RateLimitError as e:
        print(f'Blocked: {e}')
```

Expected output:

code — Code Example

```
Price of AAPL: $100
Price of AAPL: $100
Price of AAPL: $100
```

```
Blocked: Rate limit exceeded: max 3 calls per minute for fetch_price
Blocked: Rate limit exceeded: max 3 calls per minute for fetch_price
```

Student

What about `@validate_types` ?

ChatGPT • AI Assistant

🔥 EXERCISE

Practice Problem 2 — Write `@validate_types(**type_map)`

Requirements:

- Accept keyword arguments mapping parameter names to expected types:
`@validate_types(name=str, age=int)`
- At call time, check if each specified argument matches its expected type
- Raise `TypeError` with a clear message if a mismatch is found: `'age must be int, got str'`
- Works only for positional arguments that can be matched by `inspect.signature`

Starter template:

python — Code Example

```
import functools
import inspect

def validate_types(**type_map):
    """
    Usage: @validate_types(name=str, age=int)
    Validates argument types at call time.
    """
    def decorator(func):
        sig = inspect.signature(func) # Get parameter names in order

        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            # Bind args/kwargs to parameter names
            bound = sig.bind(*args, **kwargs)
            bound.apply_defaults()
```

```

        for param_name, expected_type in type_map.items():
            if param_name in bound.arguments:
                actual_value = bound.arguments[param_name]
                # TODO: Check if isinstance(actual_value,
expected_type)
                # TODO: If not, raise TypeError with a descriptive
message

            pass

        return func(*args, **kwargs)
    return wrapper
return decorator

# Test it:
@validate_types(name=str, age=int)
def create_user(name, age):
    return f'User: {name}, Age: {age}'

print(create_user('Srikanth', 28)) # Works
print(create_user('Alex', '25'))  # Should raise TypeError

```

🔗 ANALOGY

The `call_times` list in `@rate_limit` lives in the **closure** — it's shared across all calls to the decorated function but private to that one decoration. It's like a shared tally counter sitting on the function's desk.

🔗 INTERVIEW CARD

- **Frequency:** High for roles involving API development or data pipelines
- **Question:** 'How would you implement a rate limiter for a Python function without using any external libraries?'
- **Expected Answer:** Use a closure to maintain a list of call timestamps. On each call, prune timestamps older than 60 seconds and check if the count exceeds the limit.
- **Follow-up:** 'What are the thread-safety concerns with this approach?'

🔗 PRACTICE QUIZ

Concept Questions:

1. Why does the `call_times` list in `@rate_limit` need to live inside `decorator` (not inside `wrapper`)?
2. Why does `@validate_types` use `inspect.signature` rather than directly indexing `*args` ?

3. What happens if a decorated function is called with all keyword arguments — does your `@validate_types` still work?

Output Prediction Questions:

1. If `@rate_limit(calls_per_minute=2)` is applied and you call the function 3 times within 5 seconds, what happens on the third call?
2. What does `inspect.signature(func).bind(*args, **kwargs)` return for `create_user('Srikanth', 28)` ?

Debugging Question:

1. Why might this `@validate_types` implementation fail silently?

python – Code Example

```
def validate_types(**type_map):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for i, (name, t) in enumerate(type_map.items()):
                if not isinstance(args[i], t):
                    raise TypeError(f'{name} must be {t}')
            return func(*args, **kwargs)
        return wrapper
    return decorator
```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Because the list must persist across multiple calls. If it lived inside `wrapper` , it would be re-created as an empty list every time the function is called.
2. Because `inspect.signature` handles both positional and keyword arguments, and maintains correct parameter order. Direct `args` indexing breaks when callers use keyword arguments.
3. Yes — `sig.bind(*args, **kwargs)` and `bound.apply_defaults()` resolves both positional and keyword arguments into named parameters, so keyword-only calls work fine.
4. The third call raises `RateLimitError` .
5. A `BoundArguments` object where `bound.arguments = {'name': 'Srikanth', 'age': 28}` .

6. Bug: `type_map.items()` has no guaranteed order matching the function's parameter order (in Python 3.7+ dicts are ordered, but `type_map` keys may not match `*args` index positions). Also fails entirely when keyword arguments are used.

SUMMARY CARD

- Decorator factory state (counters, caches) lives in the closure between `decorator` and `wrapper`
- Always use `inspect.signature` for argument introspection — it handles both positional and keyword args
- `@validate_types` is a lightweight type-checking layer — not a replacement for Pydantic in production
- Rate limiting with timestamps is a sliding window algorithm: prune old entries, count remaining
- Thread safety is NOT handled by these simple implementations — use `threading.Lock` for concurrent use

🔍 CHECKPOINT QUESTION

If two different functions are both decorated with `@rate_limit(calls_per_minute=5)`, do they share the same `call_times` counter or have separate ones? How does the closure model explain this?

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including decorators with arguments (decorator factories), their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Decorator Factory: Three-Level Nesting Model:** A vertical flowchart with three labeled boxes stacked top to bottom. Box 1 (oute...